

HRMS + EXPIRY OS

Multi-Tenant HR Management & Document Expiry Platform

MERN Module Breakdown, Build Sequence & JWT Role-Based Auth

This document is the engineering-ready specification for the HRMS + Expiry OS SaaS platform, built on MERN (MongoDB, Express.js, React, Node.js). It defines every module the backend needs — with its own data models, REST endpoints and JWT role guards — and the exact order to build them in, so Super Admin, Admin, HR and Staff each get a working, permission-scoped dashboard from day one.

Stack	MongoDB + Express.js + React.js + Node.js (MERN), REST API, JWT auth
Roles	Super Admin → Admin (per company) → HR → Staff/Employee — each role is created by the role above it
Flagship feature	Document Expiry Engine — upload any employee/company document, set expiry, get staged auto-reminders by email
Total modules	12 modules across 6 build phases
Auth model	One login endpoint, JWT access + refresh tokens, role claim in token, route-level RBAC middleware — identical for all 4 roles

01

ROLE HIERARCHY & ACCOUNT CREATION CHAIN

Who creates whom — this is the backbone of the whole system

There is no self-signup anywhere in this system except the very first Super Admin account (seeded manually at deployment). Every other account is created by the role directly above it, and every login — regardless of role — goes through the exact same JWT login endpoint.

Role	Created by	Scope	What they see after login
Super Admin	Seeded manually at deployment (not via UI)	All companies (all tenants)	List of every registered company, plan/billing status, ability to create a new company + its first Admin
Admin (Company Admin)	Super Admin, at company registration time	Own company only	Company dashboard: create HR/Staff, configure departments, leave policy, document types, view all company reports
HR	Admin (within their own company)	Own company only	Employee records, attendance/leave approvals, payroll runs, document upload for all staff, expiry dashboard
Staff / Employee	Admin or HR (within their own company)	Own records only	Self-service: own profile, own documents + expiry status, own payslips, apply for leave, mark attendance

Golden rule

companyId is stamped onto every Admin/HR/Staff account the moment it is created, and carried inside their JWT forever after. A global Mongoose query filter uses that companyId on every single read/write in every module below, so one company can never see another company's employees, documents or payroll — except Super Admin, who has no companyId restriction.

02

MASTER BUILD ORDER

Which module should start first, second, third — and why

Build strictly top to bottom. Auth and the Notification Engine come first because the flagship Document Expiry feature — and almost every other module — depends on both.

Phase	Modules in this phase	Why it comes at this point
1 — Foundation (Week 1-2)	1. Auth & JWT Role Access 2. Notification Engine (Email/SMS)	Nothing else can be built without login/roles and a working way to send reminders. Every later module calls these two directly.
2 — Platform Setup (Week 2-3)	3. Super Admin — Company & Plan Management 4. Company Settings (departments, designations, leave policy, document types)	Establishes tenants (companies) and the configuration each company needs before any employee record can be created.
3 — Core HR Data (Week 3-5)	5. Employee Management 6. Document Management & Expiry Reminder Engine	The flagship feature. Needs Employee records and file storage to exist first, plus the Notification Engine from Phase 1 to actually send reminders.
4 — Daily Operations (Week 5-7)	7. Attendance Management 8. Leave Management	Both reference an existing Employee record and reuse the same approval-workflow pattern; build after core HR data is stable.
5 — Payroll (Week 7-9)	9. Payroll Management	Payroll pulls from Employee, Attendance and Leave, so it must be built after all three exist and are tested.
6 — Self-Service, Reports & Security (Week 9-11, continuous after)	10. Staff Self-Service Portal 11. Reports & Dashboards 12. Audit Log & Security	Self-service is a thin, scoped layer over modules 5-9. Reporting reads from every collection above, so it's built last — but audit logging should be wired in from Phase 1 onward, not bolted on at the end.

Rule of thumb

If a module needs to reference another collection by ObjectID (e.g. Payroll references Employee, Attendance and Leave), the referenced module must already be built and tested first. This is why Payroll sits in Phase 5, after Employee/Attendance/Leave all exist.

03

AUTH & JWT — SHARED BY EVERY ROLE

One login system, one token contract, four roles

Super Admin, Admin, HR and Staff all log in through the **same** `/api/auth/login` endpoint. The server checks the password, then issues a JWT that carries the user's role, companyId and permissions. The React app then shows a completely different dashboard purely based on what's inside that token — there's no separate login page or backend per role.

Password storage	bcrypt hash (cost factor 12), never stored or logged in plain text.
Access token	JWT, HS256, 15-30 minute expiry. Payload: <code>userId, role, companyId, permissions[], tokenVersion</code> .
Refresh token	Long-lived (7-30 days), stored hashed in <code>RefreshToken</code> collection, rotated on every use, revocable per device, sent as <code>httpOnly</code> secure cookie.
Login flow	POST <code>/api/auth/login</code> (email/phone + password) → validate → issue access + refresh token pair.
Role-based middleware	<code>requireAuth()</code> verifies the JWT → attaches <code>req.user</code> → <code>requireRole(['admin','hr'])</code> blocks the route if the role doesn't match.
Company scoping middleware	A single Mongoose plugin auto-filters every query by <code>req.user.companyId</code> for Admin/HR/Staff; Super Admin queries skip this filter entirely.
Password reset	Time-limited signed token emailed to the user; POST <code>/api/auth/forgot-password</code> , POST <code>/api/auth/reset-password</code> .
Forced logout	<code>tokenVersion</code> on the User document is bumped on password change, role change or account deactivation — this instantly invalidates every previously issued token for that user.

Role & permission matrix

Role	Scope	Typical JWT permissions
Super Admin	All companies (global)	<code>manage_companies</code> , <code>manage_plans</code> , <code>suspend_company</code> , <code>view_all</code> , <code>impersonate_admin</code>
Admin	Own company only	<code>manage_hr_staff</code> , <code>manage_settings</code> , <code>view_company_reports</code> , <code>approve_payroll</code>
HR	Own company only	<code>manage_employees</code> , <code>manage_attendance</code> , <code>manage_leave</code> , <code>run_payroll</code> , <code>manage_documents</code>
Staff / Employee	Own records only	<code>view_own_profile</code> , <code>apply_leave</code> , <code>mark_attendance</code> , <code>upload_own_document</code> , <code>view_own_payslip</code>

03A

MODULE CATALOGUE

All 12 modules — in build order — with full technical detail

Depends on	None — this is the foundation module
Roles involved	Super Admin, Admin, HR, Staff
Description	Central login, JWT issuing/refreshing and role/permission middleware used by every other module in this document. No public self-signup — accounts are created top-down per Section 01.

Functional requirements

- Single login endpoint for all 4 roles; the role is read from the User document, never chosen by the client.
- JWT access token (15-30 min) + rotating refresh token (httpOnly cookie).
- Password hashing with bcrypt; forced password reset on first login for newly created accounts.
- requireAuth and requireRole/requirePermission Express middleware reused by every route file below.
- Company-scoping middleware that auto-filters every query by companyId for Admin/HR/Staff.
- Account activate/deactivate; deactivation instantly revokes all active tokens (tokenVersion bump).

MongoDB / Mongoose data models

Collection	Key Fields
User	name, email, phone, passwordHash, role(super_admin/admin/hr/staff), companyId, isActive, mustResetPassword, tokenVersion, lastLoginAt
RefreshToken	userId, tokenHash, deviceInfo, ip, expiresAt, revoked

Core API endpoints

Method	Endpoint	Access (JWT role guard)
POST	/api/auth/login	Public
POST	/api/auth/refresh	Public (valid refresh cookie required)
POST	/api/auth/logout	Authenticated (any role)
GET	/api/auth/me	Authenticated (any role)
POST	/api/auth/change-password	Authenticated (any role)
POST	/api/auth/forgot-password	Public
POST	/api/auth/reset-password	Public (valid reset token)

Auth note

This module IS the JWT implementation. Every module below simply imports requireAuth() and requireRole() from here — never duplicate token logic anywhere else.

Depends on	Module 1 (Auth, for user contact info)
Roles involved	System-level service, used by every module
Description	One reusable service for Email (and optionally SMS/WhatsApp) used by the Document Expiry Engine, leave approvals, payroll notices and account-creation emails.

Functional requirements

- Central sendNotification({userId, channel, template, data}) function callable from any module.
- Template registry: welcome-email (new account + temp password), document-expiring, leave-status-changed, payslip-ready.
- Async job queue (node-cron + a queue like Agenda/BullMQ) so email sending never blocks API responses.
- Delivery status tracking (sent/failed) with retry.

MongoDB / Mongoose data models

Collection	Key Fields
NotificationTemplate	code, channel, subject, body, variables[]
NotificationLog	userId, channel, templateCode, status, sentAt, error

Core API endpoints

Method	Endpoint	Access (JWT role guard)
GET	/api/notifications/log	Admin, Super Admin
POST	/api/notifications/test-send	Super Admin (testing only)

Depends on	Modules 1, 2
Roles involved	Super Admin
Description	Super Admin registers a new company (tenant), creates its first Admin account with a temporary password, assigns a subscription plan, and can suspend/reactivate a company.

Functional requirements

- Registering a company automatically creates exactly one Admin user scoped to that companyId.
- Temporary password is generated and emailed to the new Admin via the Notification Engine; mustResetPassword = true forces a change on first login.
- Super Admin can view every registered company, its plan, employee count and active/suspended status.
- Suspending a company blocks login for every user under that companyId without deleting any data.

MongoDB / Mongoose data models

Collection	Key Fields
Company	name, planId, adminUserId, isActive, employeeLimit, createdAt
Plan	name, employeeLimit, features[], monthlyPrice

Core API endpoints

Method	Endpoint	Access (JWT role guard)
POST	/api/superadmin/companies	Super Admin only
GET	/api/superadmin/companies	Super Admin only
PATCH	/api/superadmin/companies/:id/suspend	Super Admin only
PATCH	/api/superadmin/companies/:id/plan	Super Admin only

Auth note

Every route here explicitly checks role === 'super_admin' and intentionally does NOT apply the companyId filter, since Super Admin's whole job is to see across all companies.

Depends on	Module 3
Roles involved	Admin
Description	Per-company configuration Admin sets up once: departments, designations, leave types/policy, holiday calendar and the list of document types staff can be asked to upload.

Functional requirements

- Admin defines departments and designations used when HR creates employee records.
- Admin defines leave types (sick, casual, annual) with annual quota and carry-forward rule.
- Admin defines custom document types (e.g. Citizenship, PAN, Passport, Contract, Insurance) with a default reminder schedule per type.
- Holiday calendar used by Attendance to distinguish working days from holidays.

MongoDB / Mongoose data models

Collection	Key Fields
Department	companyId, name
Designation	companyId, name, departmentId
LeaveType	companyId, name, annualQuota, carryForward
DocumentType	companyId, name, defaultReminderOffsetsDays[]
Holiday	companyId, date, name

Core API endpoints

Method	Endpoint	Access (JWT role guard)
POST	/api/settings/departments	Admin
POST	/api/settings/leave-types	Admin
POST	/api/settings/document-types	Admin
GET	/api/settings	Admin, HR (read-only)

Depends on	Modules 1, 4
Roles involved	Admin, HR
Description	Staff directory: Admin or HR creates every Staff/HR account, assigns department, designation and reporting manager. This is the record every other module (attendance, leave, payroll, documents) attaches to.

Functional requirements

- Creating an employee also creates their linked User login (role = hr or staff) with a temp password.
- Reporting-manager hierarchy used for leave-approval routing.
- Employee profile: department, designation, joining date, contact info, employment status.
- Deactivating an employee revokes their login (tokenVersion bump) without deleting historical records.

MongoDB / Mongoose data models

Collection	Key Fields
Employee	userId, companyId, departmentId, designationId, reportingManagerId, joiningDate, status(active/inactive)

Core API endpoints

Method	Endpoint	Access (JWT role guard)
POST	/api/employees	Admin, HR
GET	/api/employees	Admin, HR (all in company), Staff (own record only)
PATCH	/api/employees/:id	Admin, HR
PATCH	/api/employees/:id/deactivate	Admin, HR

Depends on	Modules 2, 4, 5
Roles involved	HR (uploads for any staff), Staff (uploads own), Admin (dashboard view)
Description	THE FLAGSHIP FEATURE. Upload any employee or company document (passport, visa, license, contract, insurance, PAN/Aadhaar, citizenship) against a chosen Document Type, set its expiry date, and let the system auto-remind everyone on a staged schedule before it lapses.

Functional requirements

- Upload a file (S3/Cloudinary signed URL) tagged with employeeId, documentTypeId and an expiryDate.
- Daily cron job scans all documents and finds ones expiring in 90/60/30/7/0 days (configurable per DocumentType from Module 4).
- Staged reminders: notify the employee first; if unresolved after N days, escalate to HR/Admin automatically — reuses Module 2, no separate email logic here.
- Color-coded expiry dashboard (red/orange/green) filterable by department and document type.
- New version replaces the old one on renewal, but full version history is kept for audit.
- Bulk upload option for HR when digitizing existing paper documents during onboarding.

MongoDB / Mongoose data models

Collection	Key Fields
Document	employeeId, companyId, documentTypeId, fileUrl, expiryDate, version, uploadedBy, status(valid/expiring/expired)
ExpiryReminderLog	documentId, sentAt, channel, escalatedTo, status

Core API endpoints

Method	Endpoint	Access (JWT role guard)
POST	/api/documents/upload-url	HR, Staff (own only)
POST	/api/documents	HR, Staff (own only)
GET	/api/documents/expiring?days=30	HR, Admin
GET	/api/documents?employeeId=	HR, Admin, Staff (own only)
PATCH	/api/documents/:id/renew	HR, Staff (own only)

Auth note

Staff can only ever query/upload documents where employeeId matches their own linked Employee record — enforced in the controller, not just hidden in the UI.

Depends on	Modules 4, 5
Roles involved	Staff (self check-in), HR (bulk/manual entry, corrections)
Description	Daily attendance capture with shift awareness and holiday calendar integration; feeds directly into Payroll in Phase 5.

Functional requirements

- Staff can check-in/check-out from the self-service portal; HR can view, correct or bulk-import attendance.
- Holiday calendar (Module 4) automatically excludes non-working days from late/absence calculations.
- Regularization requests: Staff can request a correction, HR/reporting manager approves or rejects.

MongoDB / Mongoose data models

Collection	Key Fields
Attendance	employeeId, companyId, date, checkIn, checkOut, status(present/absent/late/half-day)
RegularizationRequest	attendanceId, requestedBy, reason, status, approvedBy

Core API endpoints

Method	Endpoint	Access (JWT role guard)
POST	/api/attendance/check-in	Staff (self)
POST	/api/attendance/check-out	Staff (self)
GET	/api/attendance?employeeId=&month;=	HR, Admin, Staff (own only)
POST	/api/attendance/regularize	Staff
PATCH	/api/attendance/regularize/:id/approve	HR, reporting manager

Depends on	Modules 4, 5
Roles involved	Staff (apply), HR/reporting manager (approve)
Description	Leave application, approval routing through the reporting-manager hierarchy, and balance tracking against the leave types configured in Module 4.

Functional requirements

- Staff applies for leave against a LeaveType; system checks remaining balance before submission.
- Approval routes to the employee's reportingManagerId first, escalates to HR if no action within policy window.
- Approved/rejected leave triggers a notification to the employee (Module 2) and updates their balance.

MongoDB / Mongoose data models

Collection	Key Fields
LeaveRequest	employeeId, companyId, leaveTypeId, fromDate, toDate, reason, status, approvedBy
LeaveBalance	employeeId, leaveTypeId, year, used, remaining

Core API endpoints

Method	Endpoint	Access (JWT role guard)
POST	/api/leave-requests	Staff
GET	/api/leave-requests?employeeId=	HR, reporting manager, Staff (own only)
PATCH	/api/leave-requests/:id/approve	Reporting manager, HR
PATCH	/api/leave-requests/:id/reject	Reporting manager, HR
GET	/api/leave-balance/:employeeId	Staff (own only), HR, Admin

Depends on	Modules 5, 7, 8
Roles involved	HR (run payroll), Admin (approve), Staff (view own payslip)
Description	Salary structure per employee, monthly payroll generation pulling in attendance and approved leave, and downloadable payslips.

Functional requirements

- Salary structure: basic + allowances + deductions per employee, set by HR/Admin.
- Payroll run for a given month reads Attendance (Module 7) and approved LeaveRequests (Module 8) to compute payable days automatically.
- Generated payslip stored as a document and made available in the Staff Self-Service Portal (Module 10).
- Payroll run requires Admin approval before payslips are released to staff.

MongoDB / Mongoose data models

Collection	Key Fields
SalaryStructure	employeeId, basic, allowances[], deductions[]
Payroll	employeeId, companyId, period(YYYY-MM), payableDays, grossPay, deductions, netPay, status(draft/approved/released), payslipUrl

Core API endpoints

Method	Endpoint	Access (JWT role guard)
POST	/api/payroll/salary-structure/:employeeId	HR, Admin
POST	/api/payroll/run	HR (creates draft for the month)
PATCH	/api/payroll/:id/approve	Admin
GET	/api/payroll?employeeId=.=	Staff (own only), HR, Admin

Depends on	Modules 6, 7, 8, 9
Roles involved	Staff
Description	A single scoped dashboard where an employee sees only their own data — a thin read/write layer over the modules above, filtered to <code>employeeId = self</code> .

Functional requirements

- Profile view/edit (limited fields), own document list with expiry status and upload option.
- Apply for leave, view leave balance, check-in/check-out, view attendance history.
- View and download own payslips once released.

MongoDB / Mongoose data models

Collection	Key Fields
(no new collections — reads/writes existing ones scoped to <code>req.user.employeeId</code>)	N/A

Core API endpoints

Method	Endpoint	Access (JWT role guard)
GET	<code>/api/staff/dashboard</code>	Staff (self only)
GET	<code>/api/staff/documents</code>	Staff (self only)
GET	<code>/api/staff/payslips</code>	Staff (self only)

Auth note

Every route in this module hard-filters by `req.user.employeeId` server-side — the frontend never sends an `employeeId` the backend trusts blindly.

Depends on	All modules above
Roles involved	Super Admin, Admin, HR
Description	Aggregation layer producing role-scoped dashboards: Super Admin sees all companies, Admin/HR see their own company's headcount, attendance, leave, payroll cost and document compliance.

Functional requirements

- Super Admin dashboard: total companies, active/suspended, total employees, plan distribution.
- Admin/HR dashboard: headcount by department, attendance summary, pending leave requests, payroll cost, document compliance %.
- Filterable by date range and department; export to PDF/Excel for Admin and HR.

MongoDB / Mongoose data models

Collection	Key Fields
SavedReportView	userId, filters{}, name

Core API endpoints

Method	Endpoint	Access (JWT role guard)
GET	/api/superadmin/dashboard	Super Admin only
GET	/api/reports/company-overview	Admin, HR
GET	/api/reports/document-compliance	Admin, HR
GET	/api/reports/export?type=&format;=	Admin, HR

Depends on	Wired into every module from Phase 1 onward
Roles involved	Super Admin, Admin (own company log only)
Description	Cross-cutting hardening: who changed what and when, rate limiting, input validation and backups — applied continuously across every module, not bolted on at the end.

Functional requirements

- Global audit-log middleware records user, action, entity type/id, before/after values on every create/update/delete route.
- Helmet, express-rate-limit and Joi/Zod input validation on every route.
- Automated encrypted MongoDB + file-storage backups with periodic restore testing.
- Login history and failed-login tracking per user, visible to Admin for their own company.

MongoDB / Mongoose data models

Collection	Key Fields
AuditLog	userId, companyId, action, entityType, entityId, before{}, after{}, ip, createdAt
BackupLog	type, location, sizeBytes, status, createdAt

Core API endpoints

Method	Endpoint	Access (JWT role guard)
GET	/api/audit-logs	Super Admin (all), Admin (own company only)
GET	/api/superadmin/backups	Super Admin only

SUGGESTED MERN PROJECT STRUCTURE

Backend organized by module, matching this document

```

/hrms-backend
  /src
    /config          -> db.js, env.js, cloudStorage.js
    /middleware       -> requireAuth.js, requireRole.js, companyScope.js, auditLog.js,
errorHandler.js
    /modules
      /auth           -> auth.model.js, auth.controller.js, auth.routes.js, jwt.util.js
      /notifications  -> notification.model.js, notification.service.js, templates/
      /superadmin     -> company.controller.js, plan.controller.js
      /settings       -> department, designation, leaveType, documentType, holiday
      /employees
      /documents      -> document.model.js, expiryReminder.job.js (core feature)
      /attendance
      /leave
      /payroll
      /staff-portal
      /reports
    /jobs             -> expiryReminder.job.js, payrollReminder.job.js (node-cron / Agenda)
    /utils
    app.js
    server.js
    package.json

/hrms-frontend (React)
  /src
    /api              -> axios instance with JWT interceptor + refresh handling
    /auth             -> AuthContext, PrivateRoute, RoleGuard
    /features         -> one folder per module: superadmin/, admin/, hr/, staff/
    /shared           -> UI kit, layout, charts
  
```

Frontend JWT handling

The React app stores only the short-lived access token in memory (not localStorage, to reduce XSS risk); the refresh token lives in an httpOnly cookie. An axios response interceptor automatically calls /api/auth/refresh on a 401 and retries the original request once. The route the user lands on after login is decided purely by role in the JWT.

05

TECH STACK & NON-FUNCTIONAL REQUIREMENTS

What runs underneath every module above

Frontend	React 18+, React Router, Redux Toolkit or React Query, Tailwind/Material UI, Axios
Backend	Node.js 20 LTS, Express.js, Mongoose ODM, Joi/Zod for input validation
Database	MongoDB Atlas, indexes on companyId, employeeId, expiryDate
Auth	jsonwebtoken, bcrypt, express-rate-limit, helmet, cookie-parser (httpOnly refresh cookie)
File storage	AWS S3 or Cloudinary via signed upload URLs
Jobs/Queues	node-cron or Agenda/BullMQ (Redis) for the daily expiry-scan and payroll reminders
Email/SMS	Nodemailer/SendGrid (email); Twilio or a local SMS gateway if SMS is needed later
DevOps	Docker, CI/CD (GitHub Actions), staging + production environments, automated backups
Monitoring	Centralized logging (Winston/Pino), error tracking (Sentry)

Non-functional requirements

- Every list endpoint must be paginated and indexed — never return an unbounded collection scan.
- companyId scoping enforced at the Mongoose middleware level, not only in controllers, so a missed check in one route can't leak cross-tenant data.
- All monetary fields (payroll) stored as integers (smallest currency unit) to avoid floating-point rounding errors.
- All file uploads validated for MIME type and size before storage.
- API versioned from day one (/api/v1/...) so a future mobile app never breaks the web client.
- Target: 99.5% uptime, automated daily backups with periodic restore drills.

Final reminder

Start with Module 1 (Auth & JWT) and Module 2 (Notification Engine) before writing a single line of Employee, Document or Payroll code. Every module from Phase 2 onward assumes these two already work end-to-end for all four roles.